

AETHER v0.2 — Design Spec

Status: Draft / RFC · **Targets:** the two things reviewers will most likely challenge in v0.1 — (1) what happens when an escrow job goes wrong, and (2) the single-registry point of failure.

v0.1 shipped discovery, a four-message handshake, Ed25519-signed envelopes, and three settlement models (immediate, escrow, phased). v0.2 does **not** change that core wire format. It adds two backward-compatible extensions on top of it. Everything in v0.1 keeps working unchanged; v0.2 fields are optional and ignored by older clients.

Extension 1 — Dispute Resolution (escrow)

The problem

Today `EscrowSettlement` locks funds (`lock()`) and releases them to the provider on `settle()` . That models the happy path. It has no answer for: provider delivers garbage, provider never delivers, or requester refuses to accept good work to avoid paying. Right now the funds just sit locked, or release regardless of quality. Reviewers will hit this immediately.

Design principle

The protocol should **define the messages and the state machine**, but stay dumb about who is right. Arbitration (the judgment call) is pluggable — the protocol carries the dispute, an arbiter resolves it. This keeps the core neutral and un-opinionated, which is what lets it stay small and adoptable.

New message types

Two new signed messages, following the exact pattern of `SettlementOffer` / `AcceptanceReceipt` (a `_signable()` dict, `sign()` , `verify()`):

`DisputeClaim` → raised by either party against a locked/delivered escrow envelope
`DisputeResolution` → issued by the agreed arbiter, releasing funds per its verdict

`DisputeClaim` fields:

- `dispute_id` , `envelope_id` (the task under dispute)
- `claimant` (agent id raising it), `respondent`
- `reason` — enum: `non_delivery` | `quality` | `scope` | `non_payment` | `other`
- `evidence_hash` — SHA-256 of an off-protocol evidence blob (logs, output, transcript). Keeps large payloads off the wire while remaining tamper-evident, consistent with the envelope audit-trail approach.
- `requested_outcome` — enum: `refund` | `release` | `split`
- `opened_at` , `signature`

`DisputeResolution` fields:

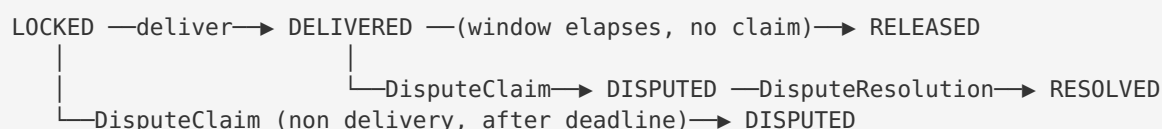
- `dispute_id` , `envelope_id`
- `arbiter` (agent id), which must match the `arbiter` agreed in the offer (see below)
- `verdict` — enum: `release_to_provider` | `refund_to_requester` | `split`
- `split_bps` — if `split` , basis points to provider (0–10000); remainder refunds requester

- `rationale_hash` — SHA-256 of the arbiter’s written reasoning
- `resolved_at`, `signature`

Agreeing on an arbiter (handshake change)

`SettlementOffer` gains one optional field: `arbiter` (agent id) and `dispute_window_secs` (how long after delivery a claim may be raised). Because it’s inside the already-signed offer, both parties cryptographically agree on who judges and how long the window is before any work starts. If `arbiter` is omitted, the job is “no-arbitration” (v0.1 behaviour) and disputes aren’t available — fully backward compatible.

Escrow state machine (v0.2)



- Funds cannot leave escrow while `DISPUTED`.
- Only the `arbiter` named in the signed offer can move `DISPUTED` → `RESOLVED`.
- `RESOLVED` splits locked funds per the verdict via the existing `Ledger.transfer`.

Code touch-points

- `aether/settlement.py` — extend `EscrowSettlement` with `open_dispute()`, `resolve_dispute()`, and explicit state tracking on the escrow record. Reuse `Ledger.transfer` for splits.
- `aether/handshake.py` — add `arbiter` + `dispute_window_secs` to `SettlementOffer._signable()` / `to_dict()` / `from_dict()`.
- New `aether/dispute.py` (~80 lines) — `DisputeClaim`, `DisputeResolution` dataclasses with sign/verify, plus an `Arbiter` protocol (interface) so anyone can plug in a rule (automated check, human, or a third-party agent discovered through AETHER itself).
- Tests: non-delivery refund, quality split, bad-faith non-payment release, and “only the agreed arbiter can resolve” rejection.

Keeps the zero-dependency, Ed25519-signed, JSON-native constraints intact.

Extension 2 — Registry Federation

The problem

v0.1 runs one registry. That’s a single point of failure and, worse for the pitch, a single point of control — which contradicts “owned by no single platform.” Reviewers will ask: what stops this from being just another centralized broker?

Design principle

Manifests are **already self-signed**. That’s the key: a manifest’s authenticity does not depend on which registry served it. So registries can freely gossip and mirror each other’s manifests without any trust between them — a client verifies the signature regardless of the source. Federation is therefore mostly a sync + dedup problem, not a consensus problem.

Model: gossip mirroring (not consensus)

Each registry keeps its own store (existing `registry_store.py`) and additionally:

1. **Peers** — a configured list of peer registry URLs.
2. **Pull sync** — periodically `GET /peer/manifests?since=<ts>` from each peer, verify every manifest's signature, and upsert into the local store.
3. **Dedup by identity** — a manifest is keyed by `agent_id`; the one with the newest signed `issued_at` wins (last-writer-wins on a signed field the agent controls). Prevents a hostile registry from resurrecting stale manifests.
4. **Origin tag** — store which peer a manifest came from, for debugging/loop-prevention; never trusted for authenticity.

No global ordering, no leader, no blockchain. Eventual consistency across the mesh, which is all discovery needs.

New endpoints (registry/server.py)

- `GET /peer/manifests?since=<unix_ts>` — returns manifests published/updated since a timestamp (for peers to pull). Signed manifests only.
- `GET /peer/info` — returns this registry's id + peer list (mesh discovery).
- Config: `AETHER_PEERS` env var (comma-separated URLs) + a background sync thread.

Client change

`RegistryClient` gains an optional list of registry URLs; `discover()` queries them in order (or all + merges), dedups by `agent_id`/newest `issued_at`, and returns the union. A client is thus never dependent on one registry being up. Single-URL usage (v0.1) is unchanged.

Anti-abuse (kept minimal)

- Reject manifests whose signature doesn't verify (already the case).
- Reject manifests with `issued_at` in the future beyond a small skew.
- Rate-limit peer pulls. Reputation remains local/advisory in v0.2 — cross-registry reputation is explicitly deferred to v0.3 (it's the genuinely hard, game-able part and shouldn't be rushed).

Code touch-points

- `aether/registry_store.py` — add `updated_since(ts)` query and last-writer-wins upsert keyed on signed `issued_at`.
- `registry/server.py` — add `/peer/manifests`, `/peer/info`, and a background pull-sync loop reading `AETHER_PEERS`.
- `aether/registry_client.py` — multi-URL discovery with merge + dedup.
- Tests: two in-process registries, publish to A, assert B mirrors it after a sync; stale-manifest rejection; signature-fail rejection on pull.

What v0.2 deliberately does NOT do

- **Cross-registry reputation** — deferred to v0.3. Local reputation stays advisory. (Reputation gaming is a whole problem; don't half-solve it.)
- **On-chain / real-money settlement** — the `Ledger` stays an abstract accounting interface. Bridging to real rails is an adapter concern, not core protocol.

- **Automated arbitration logic** — the protocol defines dispute messages and state, not verdicts. Arbiters are pluggable.

Rollout

1. Land Extension 1 (dispute) first — it's self-contained and answers the most common escrow objection.
2. Land Extension 2 (federation) second — stand up a second registry instance and demo A↔B mirroring live.
3. Tag `v0.2.0`, update README + landing page ("federated, dispute-aware"), and write a short "what's new" note to reply to launch feedback with.

Both extensions are additive and backward-compatible: a v0.1 agent and a v0.2 agent can still transact on the v0.1 happy path.